Exemplo 2: blog em Rails e em Vetra

Um blog com posts, tags, comentários moderados e área administrativa, implementado em Rails 8.1 (código real) e em Vetra (sintaxe proposta, que ainda não existe). Diferente do Exemplo 1, não é multiempresa: mostra o Vetra no caso simples.

Escopo

Um blog renderizado no servidor (HTML), com área pública e área administrativa.

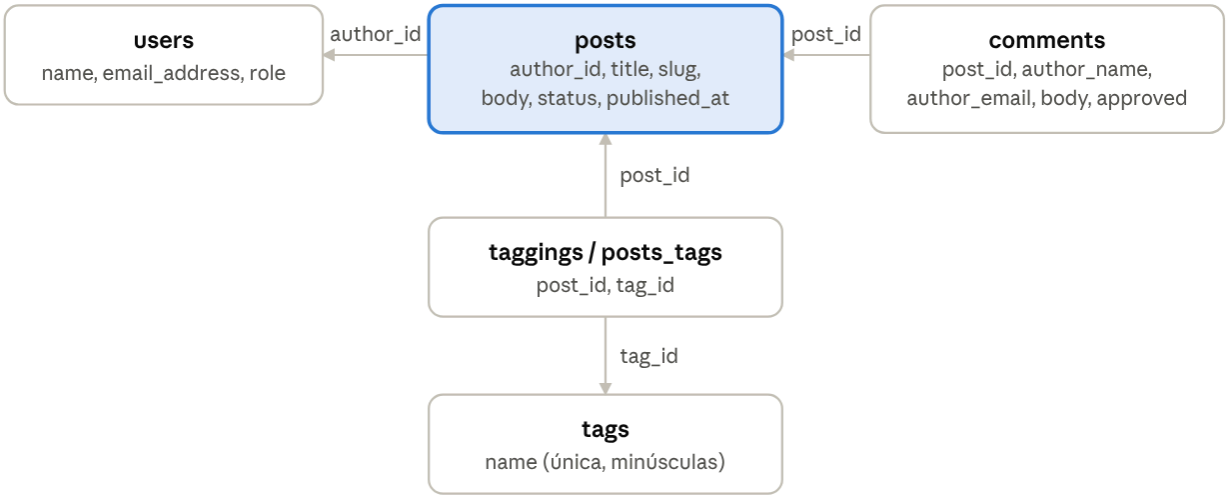
- **Área pública:** lista dos 20 posts publicados mais recentes, filtro por tag, página do post pelo slug com comentários aprovados e formulário de comentário (com limite de 5 envios por minuto).
- **Área administrativa (login):** autores criam, editam, publicam e excluem os próprios posts; o administrador gerencia todos os posts e modera comentários.
- **Post:** título, slug gerado do título, texto, status (rascunho ou publicado), data de publicação, autor e tags.
- **Comentário:** nome, e-mail (dado pessoal, nunca exibido), texto e aprovação.
- **Regra de negócio:** só rascunhos podem ser publicados; publicar grava a data de publicação.

Fora do exemplo nos dois lados: layout e CSS, paginação, texto rico (Markdown ou Action Text) e e-mails.

Arquitetura do projeto

O modelo de dados é o mesmo nos dois frameworks; no Rails são 19 arquivos escritos ou editados, organizados por tipo técnico, e no Vetra são 10, organizados pelo domínio `blog` e pela camada de UI.

Posts ligam autores, comentários e tags
Modelo de dados do Exemplo 2 · setas apontam para a tabela referenciada



modelo de dados do blog · 5 tabelas

A tabela de junção é escrita à mão no Rails (`taggings`) e gerada pelo `many_to_many :tags` no Vetra (`posts_tags`).

As árvores abaixo mostram o projeto completo, como no explorador do editor. Marcações: `# escrito` é código deste exemplo; `# editado` é arquivo gerado e alterado à mão; itens sem marcação são gerados e não alterados. Pastas sem relação com o exemplo aparecem fechadas, com uma nota do que contêm.

Estrutura em Rails 8.1

```
blog/
├── .github/                                  # CI e Dependabot
├── .kamal/                                   # hooks e segredos do deploy
├── app/
│   ├── assets/                               # imagens e CSS
│   ├── controllers/
│   │   ├── admin/
│   │   │   ├── comments_controller.rb        # escrito
│   │   │   └── posts_controller.rb           # escrito
│   │   ├── concerns/
│   │   │   └── authentication.rb             # gerado pela autenticação
│   │   ├── application_controller.rb
│   │   ├── comments_controller.rb            # escrito
│   │   ├── passwords_controller.rb           # gerado pela autenticação
│   │   ├── posts_controller.rb               # escrito
│   │   └── sessions_controller.rb            # gerado pela autenticação
│   ├── helpers/
│   ├── javascript/                           # Turbo e Stimulus via importmap
```

```
|   ├── jobs/
|   ├── mailers/
|   |   ├── application_mailer.rb
|   |   └── passwords_mailer.rb           # gerado pela autenticação
|   ├── models/
|   |   ├── concerns/
|   |   ├── application_record.rb
|   |   ├── comment.rb                    # escrito
|   |   ├── current.rb                    # gerado pela autenticação
|   |   ├── post.rb                       # escrito
|   |   ├── session.rb                    # gerado pela autenticação
|   |   ├── tag.rb                        # escrito
|   |   ├── tagging.rb                    # escrito
|   |   └── user.rb                       # editado
|   └── views/
|       ├── admin/
|       |   ├── comments/
|       |   |   └── index.html.erb        # escrito
|       |   └── posts/
|       |       ├── _form.html.erb        # escrito
|       |       ├── edit.html.erb         # escrito
|       |       ├── index.html.erb        # escrito
|       |       └── new.html.erb          # escrito
|       ├── layouts/
|       |   └── application.html.erb
|       ├── passwords/                    # gerado pela autenticação
|       ├── passwords_mailer/             # gerado pela autenticação
|       ├── posts/
|       |   ├── index.html.erb            # escrito
|       |   └── show.html.erb             # escrito
|       ├── pwa/
|       └── sessions/                     # gerado pela autenticação
├── bin/                                  # rails, rake, setup, dev, jobs,
kamal, ci
├── config/
|   ├── environments/
|   ├── initializers/
|   ├── locales/
|   ├── application.rb                    # editado: idioma pt-BR
|   ├── boot.rb
|   ├── cable.yml
|   ├── cache.yml
|   ├── ci.rb
|   ├── credentials.yml.enc
|   ├── database.yml
|   ├── deploy.yml
```

```
|   ├── environment.rb
|   ├── importmap.rb
|   ├── master.key
|   ├── puma.rb
|   ├── queue.yml
|   ├── recurring.yml
|   ├── routes.rb                          # editado
|   └── storage.yml
├── db/
|   ├── migrate/
|   |   ├── ..._create_users.rb            # gerado pela autenticação
|   |   ├── ..._create_sessions.rb         # gerado pela autenticação
|   |   └── 20261029100000_create_blog.rb  # escrito
|   ├── cable_schema.rb
|   ├── cache_schema.rb
|   ├── queue_schema.rb
|   ├── schema.rb
|   └── seeds.rb
├── lib/
|   └── tasks/
├── log/
├── public/                                # páginas de erro, ícones,
robots.txt
├── script/
├── storage/
├── test/                                  # controllers, models, system,
fixtures
├── tmp/
├── vendor/
├── .dockerignore
├── .gitattributes
├── .gitignore
├── .rubocop.yml
├── .ruby-version
├── config.ru
├── Dockerfile
├── Gemfile                                # alterado pelo bundle add
├── Gemfile.lock
├── Rakefile
└── README.md
```

Estrutura em Vetra

```
blog/
├── .vetra/
│   ├── cache/
│   └── graph.json                    # IR versionada do grafo
├── app/
│   ├── domains/
│   │   └── blog/
│   │       ├── approve_comment.rb    # escrito · command
│   │       ├── comment.rb            # escrito · entity
│   │       ├── post.rb               # escrito · entity
│   │       ├── publish_post.rb       # escrito · command
│   │       ├── queries.rb            # escrito · query
│   │       ├── roles.rb              # escrito · roles
│   │       └── tag.rb                # escrito · entity
│   └── ui/
│       ├── assets/                   # CSS e imagens
│       ├── layouts/
│       │   └── application.html.erb
│       ├── templates/
│       │   └── posts/
│       │       ├── index.html.erb    # escrito
│       │       └── show.html.erb     # escrito
│       └── pages.rb                  # escrito
├── bin/
│   ├── setup
│   └── vetra
├── config/
│   ├── environments/
│   │   ├── development.rb
│   │   ├── production.rb
│   │   └── test.rb
│   ├── locales/
│   │   └── pt-BR.yml
│   ├── app.rb                        # idioma, auth, ui
│   ├── database.rb                   # papéis do banco: aplicação e
migração
│   ├── secrets.enc                   # segredos cifrados
│   └── server.rb                     # Falcon
├── db/
│   ├── plans/                        # planos de migração expand/contract
│   ├── schema.sql                    # retrato do banco
│   └── seeds.rb
├── lib/
├── log/
```

```
├── public/
├── storage/
├── test/
│   ├── domains/
│   │   └── blog/                    # testes escritos pela equipe
│   ├── generated/                   # testes de autorização gerados do
grafo
│   └── test_helper.rb
├── tmp/
├── .gitignore
├── .ruby-version                    # 4.0.9
├── config.ru
├── Dockerfile
├── Gemfile
├── Gemfile.lock
└── README.md
```

Usuários, sessões, login e telas administrativas não aparecem na árvore do Vetra porque são gerados pela opção `--auth=password` e por `crud ui: :admin`.

Rails 8.1 — criação do projeto, banco de dados e tabelas

O gerador de autenticação do Rails 8 cria `User`, `Session`, login, recuperação de senha e o concern `Authentication`; o restante abaixo é escrito à mão.

```
rails new blog --database=postgresql
cd blog
bundle add rails-i18n
bin/rails generate authentication
bin/rails generate migration CreateBlog
bin/rails db:create db:migrate
```

```
# config/application.rb (linha adicionada dentro da classe Application)
config.i18n.default_locale = :"pt-BR"
```

```
# db/migrate/20261029100000_create_blog.rb
class CreateBlog < ActiveRecord::Migration[8.1]
  def change
    add_column :users, :name, :string, null: false, default: ""
    add_column :users, :role, :string, null: false, default: "autor"

    create_table :posts do |t|
      t.references :author, null: false, foreign_key: { to_table: :users }
      t.string :title, null: false
      t.string :slug, null: false, index: { unique: true }
```

```
      t.text :body, null: false
      t.string :status, null: false, default: "draft"
      t.datetime :published_at
      t.timestamps
      t.index %i[status published_at]
    end

    create_table :comments do |t|
      t.references :post, null: false, foreign_key: true
      t.string :author_name, null: false
      t.string :author_email, null: false
      t.text :body, null: false
      t.boolean :approved, null: false, default: false
      t.timestamps
    end

    create_table :tags do |t|
      t.string :name, null: false, index: { unique: true }
      t.timestamps
    end

    create_table :taggings do |t|
      t.references :post, null: false, foreign_key: true
      t.references :tag, null: false, foreign_key: true
      t.index %i[post_id tag_id], unique: true
    end
  end
end
```

Rails 8.1 — models

Slug, tags como texto separado por vírgula e a regra de publicação ficam no model, no estilo idiomático do Rails.

```
# app/models/user.rb (gerado; linhas marcadas foram adicionadas)
class User < ApplicationRecord
  has_secure_password
  has_many :sessions, dependent: :destroy
  has_many :posts, foreign_key: :author_id, inverse_of: :author # adicionada

  normalizes :email_address, with: ->(e) { e.strip.downcase }
  validates :role, inclusion: { in: %w[admin autor] }          # adicionada

  def admin? = role == "admin"                                  # adicionada
end
```

```
# app/models/post.rb
class Post < ApplicationRecord
  belongs_to :author, class_name: "User"
  has_many :comments, dependent: :destroy
  has_many :taggings, dependent: :destroy
  has_many :tags, through: :taggings

  enum :status, %w[draft published].index_by(&:itself),
       default: "draft", prefix: true

  validates :title, presence: true, length: { maximum: 200 }
  validates :body, presence: true
  validates :slug, presence: true, uniqueness: true

  before_validation :set_slug, on: :create

  scope :visible, -> {
    status_published.where(published_at: ..Time.current)
                    .order(published_at: :desc)
  }

  def to_param = slug

  def tag_list = tags.map(&:name).join(", ")

  def tag_list=(names)
    self.tags = names.to_s.split(",").map(&:strip).reject(&:blank?).uniq
                     .map { |name| Tag.find_or_create_by!(name: name) }
  end

  def publish!
    return false unless status_draft?

    update!(status: "published", published_at: Time.current)
  end

  private

  def set_slug
    self.slug = title.to_s.parameterize if slug.blank?
  end
end
```

```
# app/models/comment.rb
class Comment < ApplicationRecord
  belongs_to :post

  validates :author_name, presence: true, length: { maximum: 80 }
  validates :author_email, format: { with: URI::MailTo::EMAIL_REGEXP }
  validates :body, presence: true, length: { maximum: 2_000 }

  scope :approved, -> { where(approved: true).order(:created_at) }
  scope :pending,  -> { where(approved: false).order(:created_at) }
end
```

```
# app/models/tag.rb
class Tag < ApplicationRecord
  has_many :taggings, dependent: :destroy
  has_many :posts, through: :taggings

  normalizes :name, with: ->(name) { name.strip.downcase }
  validates :name, presence: true, uniqueness: true
end
```

```
# app/models/tagging.rb
class Tagging < ApplicationRecord
  belongs_to :post
  belongs_to :tag
end
```

Dois títulos iguais geram o mesmo slug; a validação de unicidade recusa o segundo post, e o autor precisa mudar o título.

Rails 8.1 — controllers e rotas

O gerador de autenticação exige login em todos os controllers por padrão; os públicos liberam com `allow_unauthenticated_access`. A autorização é feita à mão: o autor só enxerga os próprios posts, e só o administrador modera comentários.

```
# app/controllers/posts_controller.rb
class PostsController < ApplicationController
  allow_unauthenticated_access

  def index
    @posts = Post.visible.includes(:author, :tags).limit(20)
    if params[:tag].present?
      @posts = @posts.joins(:tags).where(tags: { name: params[:tag] })
    end
  end

  def show
    @post = Post.visible.find_by!(slug: params[:slug])
    @comments = @post.comments.approved
    @comment = Comment.new
  end
end
```

```
# app/controllers/comments_controller.rb
class CommentsController < ApplicationController
  allow_unauthenticated_access
  rate_limit to: 5, within: 1.minute, only: :create

  def create
    @post = Post.visible.find_by!(slug: params[:post_slug])
    @comment = @post.comments.new(comment_params)

    if @comment.save
      redirect_to post_path(@post), notice: "Comentário enviado para
moderação."
    else
      @comments = @post.comments.approved
      render "posts/show", status: :unprocessable_content
    end
  end

  private

  def comment_params
    params.expect(comment: [:author_name, :author_email, :body])
  end
end
```

```
# app/controllers/admin/posts_controller.rb
module Admin
  class PostsController < ApplicationController
    before_action :set_post, only: %i[edit update destroy publish]

    def index
      @posts = scope.order(updated_at: :desc)
    end

    def new
      @post = Current.user.posts.new
    end

    def create
      @post = Current.user.posts.new(post_params)
      if @post.save
        redirect_to edit_admin_post_path(@post), notice: "Post criado."
      else
        render :new, status: :unprocessable_content
      end
    end

    def edit
    end

    def update
      if @post.update(post_params)
        redirect_to edit_admin_post_path(@post), notice: "Post atualizado."
      else
        render :edit, status: :unprocessable_content
      end
    end

    def destroy
      @post.destroy!
      redirect_to admin_posts_path, notice: "Post excluído."
    end

    def publish
      if @post.publish!
        redirect_to post_path(@post), notice: "Post publicado."
      else
        redirect_to edit_admin_post_path(@post), alert: "Só rascunhos podem
ser publicados."
      end
    end
```

```
    private

    def scope
      Current.user.admin? ? Post.all : Current.user.posts
    end

    def set_post
      @post = scope.find_by!(slug: params[:id])
    end

    def post_params
      params.expect(post: [:title, :body, :tag_list])
    end
  end
end
```

```
# app/controllers/admin/comments_controller.rb
module Admin
  class CommentsController < ApplicationController
    before_action :require_admin
    before_action :set_comment, only: %i[approve destroy]

    def index
      @comments = Comment.pending.includes(:post)
    end

    def approve
      @comment.update!(approved: true)
      redirect_to admin_comments_path, notice: "Comentário aprovado."
    end

    def destroy
      @comment.destroy!
      redirect_to admin_comments_path, notice: "Comentário excluído."
    end

    private

    def set_comment
      @comment = Comment.find(params[:id])
    end

    def require_admin
      redirect_to root_path, alert: "Acesso negado." unless
```

```
Current.user.admin?
    end
  end
end
```

```
# config/routes.rb (as duas primeiras rotas vêm do gerador)
Rails.application.routes.draw do
  resource :session
  resources :passwords, param: :token

  root "posts#index"
  get "tags/:tag", to: "posts#index", as: :tag

  resources :posts, only: %i[index show], param: :slug do
    resources :comments, only: :create
  end

  namespace :admin do
    resources :posts, except: :show do
      post :publish, on: :member
    end
    resources :comments, only: %i[index destroy] do
      patch :approve, on: :member
    end
  end
end
```

Rails 8.1 — views

Sete templates ERB: dois públicos e cinco administrativos. As mensagens de `notice` e `alert` são exibidas no layout gerado.

```
<%# app/views/posts/index.html.erb %>
<h1><%= params[:tag] ? "Posts com a tag #{params[:tag]}" : "Blog" %></h1>

<% @posts.each do |post| %>
  <article>
    <h2><%= link_to post.title, post_path(post) %></h2>
    <p><%= post.author.name %> · <%= l(post.published_at, format: :long) %>
</p>
    <p><%= truncate(post.body, length: 280) %></p>
    <p>
      <% post.tags.each do |tag| %>
        <%= link_to "##{tag.name}", tag_path(tag.name) %>
      <% end %>
    </p>
  </article>
<% end %>
```

```
<%# app/views/posts/show.html.erb %>
<article>
  <h1><%= @post.title %></h1>
  <p><%= @post.author.name %> · <%= l(@post.published_at, format: :long) %>
</p>
  <%= simple_format(@post.body) %>
</article>

<section>
  <h2>Comentários (<%= @comments.size %>)</h2>
  <% @comments.each do |comment| %>
    <div>
      <strong><%= comment.author_name %></strong>
      <%= simple_format(comment.body) %>
    </div>
  <% end %>

  <h3>Deixe um comentário</h3>
  <%= form_with model: [@post, @comment] do |f| %>
    <% @comment.errors.full_messages.each do |message| %>
      <p><%= message %></p>
    <% end %>
    <%= f.text_field :author_name, placeholder: "Nome" %>
    <%= f.email_field :author_email, placeholder: "E-mail (não será
publicado)" %>
```

```
    <%= f.text_area :body, rows: 4 %>
    <%= f.submit "Enviar" %>
  <% end %>
</section>
```

```
<%# app/views/admin/posts/index.html.erb %>
<h1>Posts</h1>
<%= link_to "Novo post", new_admin_post_path %>

<table>
  <% @posts.each do |post| %>
    <tr>
      <td><%= link_to post.title, edit_admin_post_path(post) %></td>
      <td><%= post.status %></td>
      <td>
        <% if post.status_draft? %>
          <%= button_to "Publicar", publish_admin_post_path(post) %>
        <% end %>
        <%= button_to "Excluir", admin_post_path(post), method: :delete,
                      form: { data: { turbo_confirm: "Excluir este post?" } }
%>
      </td>
    </tr>
  <% end %>
</table>
```

```
<%# app/views/admin/posts/_form.html.erb %>
<%= form_with model: [:admin, post] do |f| %>
  <% post.errors.full_messages.each do |message| %>
    <p><%= message %></p>
  <% end %>
  <%= f.label :title, "Título" %>
  <%= f.text_field :title %>
  <%= f.label :body, "Texto" %>
  <%= f.text_area :body, rows: 16 %>
  <%= f.label :tag_list, "Tags (separadas por vírgula)" %>
  <%= f.text_field :tag_list %>
  <%= f.submit "Salvar" %>
<% end %>
```

```
<%# app/views/admin/posts/new.html.erb %>
<h1>Novo post</h1>
<%= render "form", post: @post %>
```

```
<%# app/views/admin/posts/edit.html.erb %>
<h1>Editar post</h1>
<%= render "form", post: @post %>
```

```
<%# app/views/admin/comments/index.html.erb %>
<h1>Comentários pendentes</h1>

<% @comments.each do |comment| %>
  <div>
    <p>
      <strong><%= comment.author_name %></strong> em
      <%= link_to comment.post.title, post_path(comment.post) %>
    </p>
    <%= simple_format(comment.body) %>
    <%= button_to "Aprovar", approve_admin_comment_path(comment), method:
:patch %>
    <%= button_to "Excluir", admin_comment_path(comment), method: :delete %>
  </div>
<% end %>
```

Vetra — criação do projeto, banco de dados e entidades

Sintaxe proposta: nada abaixo existe ainda. Sem `--tenant`, o projeto não é multiempresa e não há RLS por tenant — o caso simples continua simples. As tabelas não são escritas à mão: o `vetra db:plan` as deriva das entidades.

```
vetra new blog --locale=pt-BR --auth=password --ui=server
cd blog
vetra db:create
vetra db:plan      # mostra o SQL gerado a partir das entidades
vetra db:migrate
vetra dev
```

Arquivo gerado pelo `vetra new`, não escrito à mão:

```
# config/app.rb
Vetra.app :blog do
  locale "pt-BR"
  auth   :password, roles: %i[admin autor]
  ui     :server
end
```

A opção `--auth=password` gera usuários com nome e papel, sessões, login e recuperação de senha.

```
# app/domains/blog/post.rb
entity :post do
  belongs_to :author, entity: :user, default: :current_actor

  string    :title,  required: true, max: 200
  slug      :slug,   from: :title, unique: true
  text      :body,   required: true
  enum      :status, %w[draft published], default: "draft",
                     guarded: { "published" => :publish_post }
  timestamp :published_at, set_by: :publish_post
  many_to_many :tags

  crud scope: :author, ui: :admin
  index :status, :published_at
end
```

```
# app/domains/blog/comment.rb
entity :comment do
  belongs_to :post

  string  :author_name,  required: true, max: 80
  email   :author_email, required: true, personal: true
  text    :body,         required: true, max: 2_000
  boolean :approved,     default: false, set_by: :approve_comment

  crud ui: :admin
end
```

```
# app/domains/blog/tag.rb
entity :tag do
  string :name, required: true, unique: true, normalize: :downcase
end
```

O que essas três declarações geram:

- Tabelas `posts`, `comments`, `tags` e a tabela de junção `posts_tags`, com chaves estrangeiras e índices.
- Slug derivado do título, com sufixo numérico automático em caso de colisão.
- `author` preenchido com o usuário logado na criação.
- Comandos e consultas CRUD, telas administrativas de posts e comentários (lista, formulário, exclusão), com campo de tags.
- `author_email` mascarado em logs, traces e erros.

Campos de transição (ADR-021): `status: "published"` e `published_at` só mudam por `publish_post`, e `approved` só por `approve_comment`. O CRUD e as telas administrativas geradas omitem esses campos; sem isso, um autor publicaria pelo formulário de edição sem gravar a data de publicação.

Vetra — papéis, consultas e comandos

Os papéis substituem os `before_action` e os escopos escritos à mão no Rails; as consultas alimentam as páginas públicas, e os comandos cobrem as duas regras além do CRUD.

```
# app/domains/blog/roles.rb
roles do
  visitor do
    can :read,   on: :post,    where: { status: "published" }
    can :read,   on: :comment, where: { approved: true }
    can :create, on: :comment, fields: %i[post_id author_name author_email
body],
                 rate_limit: "5/min"
  end

  role :autor do
    can :manage, on: :post, scope: :author
  end

  role :admin do
    can :manage, on: %i[post comment tag]
  end
end
```

```
# app/domains/blog/queries.rb
query :published_posts do
  input :tag, :string, optional: true
  uses :posts

  call do |input, posts|
    list = posts.where(status: "published")
                .order(published_at: :desc).limit(20)
                .with(:author, :tags)
    input.tag ? list.where(tags: { name: input.tag }) : list
  end
end

query :post_by_slug do
```

```
  input :slug, :string
  uses :posts, :comments

  call do |input, posts, comments|
    post = posts.find_by!(slug: input.slug, status: "published")
    {
      post: post,
      comments: comments.where(post_id: post.id, approved:
true).order(:created_at)
    }
  end
end
```

```
# app/domains/blog/publish_post.rb
command :publish_post do
  input :post_id, :id
  authorize :update, :post, record: :post_id
  uses :posts, :clock

  call do |input, posts, clock|
    post = posts.lock!(input.post_id)
    fail! :status, "só rascunhos podem ser publicados" unless post.status ==
"draft"
    posts.update!(post, status: "published", published_at: clock.now)
  end

  expose ui: { button: "Publicar", on: :post, when: { status: "draft" } }
end
```

```
# app/domains/blog/approve_comment.rb
command :approve_comment do
  input :comment_id, :id
  authorize :update, :comment, record: :comment_id
  uses :comments

  call { |input, comments| comments.update!(input.comment_id, approved: true)
}

  expose ui: { button: "Aprovar", on: :comment, when: { approved: false } }
end
```

Implícito pelo framework:

- O visitante não consegue enviar `approved`, porque o papel limita os campos.

- A criação de comentário verifica se o post referenciado é visível para quem comenta, ou seja, se está publicado.
- O relógio vem da capacidade `clock`, o que torna a publicação testável com data fixa.
- `expose ui:` coloca os botões "Publicar" e "Aprovar" nas telas administrativas geradas, só quando a condição vale.

Vetra — páginas e templates

As páginas públicas ligam uma rota a uma consulta e a um template ERB; as telas administrativas não aparecem aqui porque são geradas por `crud ui: :admin`. Nos templates, `input` é a entrada da consulta e `result` é o que ela devolveu.

```
# app/ui/pages.rb
page "/",            query: :published_posts, template: "posts/index"
page "/tags/:tag",   query: :published_posts, template: "posts/index"
page "/posts/:slug", query: :post_by_slug,    template: "posts/show"
```

```
<%# app/ui/templates/posts/index.html.erb %>
<h1><%= input.tag ? "Posts com a tag #{input.tag}" : "Blog" %></h1>

<% result.each do |post| %>
  <article>
    <h2><%= link_to post.title, "/posts/#{post.slug}" %></h2>
    <p><%= post.author.name %> · <%= l(post.published_at, format: :long) %>
</p>
    <p><%= truncate(post.body, length: 280) %></p>
    <p>
      <% post.tags.each do |tag| %>
        <%= link_to "##{tag.name}", "/tags/#{tag.name}" %>
      <% end %>
    </p>
  </article>
<% end %>
```

```
<%# app/ui/templates/posts/show.html.erb %>
<% post = result[:post] %>
<article>
  <h1><%= post.title %></h1>
  <p><%= post.author.name %> · <%= l(post.published_at, format: :long) %></p>
  <%= simple_format(post.body) %>
</article>

<section>
```

```
  <h2>Comentários (<%= result[:comments].size %>)</h2>
  <% result[:comments].each do |comment| %>
    <div>
      <strong><%= comment.author_name %></strong>
      <%= simple_format(comment.body) %>
    </div>
  <% end %>

  <h3>Deixe um comentário</h3>
  <%= command_form :create_comment, values: { post_id: post.id },
                   success: { notice: "Comentário enviado para moderação." }
do |f| %>
    <%= f.errors %>
    <%= f.text_field :author_name, placeholder: "Nome" %>
    <%= f.email_field :author_email, placeholder: "E-mail (não será
publicado)" %>
    <%= f.text_area :body, rows: 4 %>
    <%= f.submit "Enviar" %>
  <% end %>
</section>
```

`command_form` monta o formulário a partir do contrato do comando `create_comment` (gerado pelo CRUD): valida a entrada, aplica o limite de taxa do papel, reexibe a página com os erros em caso de falha e redireciona com a mensagem em caso de sucesso.

Comparação

O Rails pede cerca de 350 linhas em 19 arquivos; o Vetra, cerca de 145 linhas em 10 arquivos. A diferença é bem menor que no Exemplo 1, porque sem multiempresa as garantias de segurança do Vetra pesam menos.

Parte	Rails 8.1 (linhas aprox.)	Vetra (linhas aprox.)
Banco de dados e tabelas	40 (migração)	0 (derivado das entidades)
Models / entidades	70	30
Autorização	dentro dos controllers	17 (papéis)
Controllers e rotas / consultas, comandos e páginas	145	55
Views públicas	42	45

Parte	Rails 8.1 (linhas aprox.)	Vetra (linhas aprox.)
Views administrativas	52	0 (geradas)
Total	**~350 em 19 arquivos**	**~145 em 10 arquivos**

Leitura honesta:

- **De onde vem o ganho:** tabelas derivadas das entidades, autorização declarativa em vez de `before_action` e escopos manuais, e telas administrativas geradas.
- **Onde não há ganho:** os templates públicos têm o mesmo tamanho nos dois. O Vetra não simplifica a camada de apresentação.
- **Admin gerado é genérico:** personalizar o visual exige "ejetar" as telas, e a vantagem diminui. O Rails também tem `bin/rails generate scaffold`, que gera views, mas sem namespace administrativo nem escopo por autor.
- **O que o Rails já faz bem e o Vetra precisa igualar:** gerador de autenticação, `rate_limit` nativo nos controllers e filtragem de e-mail nos logs de parâmetros por padrão.
- **Lacuna da proposta:** este exemplo usa uma camada de UI no servidor (`page`, templates, `command_form`, `crud ui: :admin`, `expose ui:`) que a proposta técnica ainda não especifica. Ela precisa de um ADR próprio antes de ser considerada parte do Vetra.